Sample-Efficient MAPPO for Physics-in-the-Loop Multi-Robot Training: Diagnosing and Correcting Entropy Collapse Under a Thousand-Step, Not Million-Step, Budget

Harsh Pandhe[1]

[1]Independent Research , harshpandhehome@gmail.com

Abstract

Multi-agent reinforcement learning (MARL) sample-efficiency work overwhelmingly assumes access to a fast, often GPU-parallelized simulator, where the practical constraint is wall-clock per environment step, not the number of steps a single rollout worker can collect. This assumption breaks down for physics-in-the-loop robotics training against a general-purpose simulator (Gazebo Sim) with no massively-parallel rollout path: each environment step costs real wall-clock time regardless of parallelization strategy, and a training budget of thousands, not millions, of environment steps is a hard practical ceiling, not a simplifying choice. We report a directly observed failure mode under this constraint – a centralized-critic MAPPO policy for cooperative multi-robot area coverage, trained for 4,500 total environment steps (15 iterations, train batch size 300), whose policy entropy collapsed to -0.53 by the final iteration while episode returns swung between -227 and $+3$ within the same 3–4 sampled episodes – and diagnose it as a direct consequence of leaving RLlib's advantage-estimation ($\lambda = 1.0$) and exploration-pressure (entropy_coeff $= 0.0$) hyperparameters at defaults tuned for much larger batch regimes. We derive a specific, mechanistically justified correction (reduced GAE λ, a nonzero entropy bonus, reward rebalancing to reduce per-episode return variance, and batch/episode-length adjustments that increase distinct-episode count per iteration without increasing wall-clock), implement it in a public codebase, and report the diagnosis, the fix, and its expected mechanism in full; we could not complete the corrected training run's full empirical validation in this revision due to an unrelated infrastructure fault (a segmentation fault in the local PyTorch/CUDA

installation, confirmed independent of the training code by three reproducible crash attempts) and report this transparently as an open item alongside the diagnosis and proposed fix.

Keywords: multi-agent reinforcement learning, sample efficiency, MAPPO, proximal policy optimization, robotics, entropy collapse

1 Introduction

Sample efficiency in deep reinforcement learning is usually studied under an implicit assumption: environment steps are cheap enough, or parallelizable enough, that the real constraint is algorithmic (how much a policy improves per gradient update) rather than infrastructural (how many environment steps can physically be collected before an experiment's wall-clock budget is spent). GPU-accelerated physics simulators built for exactly this assumption – running thousands of environment instances concurrently – have become the default substrate for sample-efficiency research in robot learning. This is a reasonable choice when the target deployment simulator supports it. It is not a universal property of robotics simulation: general-purpose robotics simulators used for realistic sensor modeling, ROS integration, and physically detailed multi-robot interaction (collision geometry, LiDAR ray-casting, differential-drive dynamics) are frequently run as a single real-time (or near-real-time) instance, because the fidelity that makes them useful for the target application is exactly what makes naive massive parallelization impractical without dedicated infrastructure investment most individual researchers and small teams do not have. Under this constraint, a training budget of thousands of environment steps – not the millions typical of parallelized-simulator MARL benchmarks – is not a choice, it is the wall-clock-bounded ceiling.

We report a concrete instance of what happens to a standard MARL algorithm (MAPPO [4] with a centralized critic, CTDE) under exactly this constraint, diagnose the failure mechanistically rather than empirically alone, and derive a correction from that mechanism. Our contribution is not a new algorithm; it is a grounded case study connecting a specific, reproducible failure (entropy collapse under a tiny, wall-clock-bounded sample budget) to specific hyperparameter and reward-design choices, in a setting – physics-in-the-loop multi-robot training with no fast-simulator escape hatch – that the dominant sample-efficiency literature does not directly address.

2 Related Work

Yu et al. [4] establish MAPPO with a centralized critic as a strong, simple CTDE baseline for cooperative multi-agent games, and is the algorithm family we build on; their sample-efficiency results, like most of the literature that follows, are reported in a regime with orders of magnitude more environment steps than are available under our constraint. Sample-efficiency techniques aimed explicitly at closing this gap increasingly rely on differentiable simulation or model-based rollouts to reduce real-environment sample requirements by large factors [5, 3] – a complementary but architecturally distinct strategy from the hyperparameter- and reward-design-level correction we study, and one that requires a differentiable or learned dynamics model not assumed available here. Masked reconstruction and other representation-learning approaches improve MARL sample efficiency by extracting more training signal per environment step [1], a strategy compatible with and potentially complementary to the correction we propose, though we do not implement it here. Physics-informed MARL for distributed multi-robot problems [2] incorporates physical structure directly into the learning problem, again a complementary direction. None of this literature, to our knowledge, specifically characterizes the entropy collapse and advantage-variance failure mode we report as a direct consequence of applying default single-agent PPO hyperparameters unmodified in a small-batch, physics-in-the-loop multi-agent setting.

3 System and Training Setup

The learned system is a shared MAPPO policy for a three-robot TurtleBot3 swarm performing cooperative area coverage in Gazebo Sim (ROS 2 Jazzy), trained via Ray RLlib. The actor is a permutation-invariant network: ego features (28-dim: LiDAR, goal, velocity) are concatenated with a masked mean-pooled embedding of up to 9 neighbors' relative $(\text{distance}, \text{angle})$, each processed by a shared two-layer MLP. The critic is centralized: it additionally consumes a masked mean-pooled embedding of all teammates' joint (observation, action) pairs, following the standard CTDE recipe. Training uses `num_env_runners=0` (Gazebo must run in-process; RLlib's usual multi-worker parallel-rollout path is unavailable), so every environment step is collected serially against real-time (or near-real-time) physics – the exact constraint motivating this paper.

Baseline configuration (the diagnosed failure). 15 training iterations, train batch size 300, minibatch size 64, 5 epochs per iteration, learning rate 1×10^{-4}, PPO clip 0.2, discount $\gamma = 0.99$, episode length (`max_steps`) 150, `rollout_fragment_length` 150. Critically, GAE λ and `entropy_coeff` were left at RLlib's PPO defaults (1.0 and 0.0 respectively)

– i.e., no advantage-variance reduction and no exploration bonus. Total training budget: $15 \times 300 = 4{,}500$ environment steps.

4 Diagnosis

With train batch size 300 split across 3 agents over episodes up to 150 steps, a single training iteration collects on the order of 3–4 complete episodes. Two consequences follow directly from the baseline configuration:

(1) High-variance advantage estimates. GAE $\lambda = 1.0$ corresponds to the (unbiased but high-variance) Monte-Carlo return estimator. With only 3–4 episodes contributing to a batch, the resulting advantage estimates are dominated by whichever few episodes happened to be sampled, rather than reflecting the policy's average behavior. We observed this directly: episode returns within a single iteration's batch ranged from -227 to $+3$ – a swing far larger than plausible in-policy variation, indicative of an estimator amplifying a small sample rather than measuring a stable signal.

(2) No counter-pressure against premature convergence. `entropy_coeff`$= 0.0$ means the PPO objective contains no term rewarding continued exploration. Combined with (1) – a high-variance, noisy advantage signal – the policy has every incentive to converge quickly toward whatever locally low-variance (e.g. near-stationary, "safe") behavior it stumbles into early, since that behavior at least yields a consistent, low-magnitude return, rather than continuing to explore into the noisy, sparsely-rewarded regions of the state space where the actual task reward (area coverage) lives. We observed this directly: policy entropy fell to -0.53 by the final (15th) iteration, having been positive earlier in training – a collapse toward a low-entropy, near-deterministic policy well before 4,500 environment steps could plausibly be sufficient to have converged on a genuinely good policy for a 3-robot coordination task.

A third, compounding factor is reward design: the per-step reward combines a small dense progress/coverage term with sparse ± 20 spikes for goal-reaching and collision. Over an episode as short as 150 steps (or fewer, if the episode terminates early on collision), a single -20 collision event can dominate the cumulative return, further inflating the return variance described in (1) independent of the λ setting alone.

5 Proposed Correction

We derive a correction directly from the mechanisms in Section 4, rather than a general hyperparameter sweep:

- **GAE $\lambda : 1.0 \rightarrow 0.95$.** Directly targets mechanism (1): trades a small amount of estimator bias for a substantial reduction in advantage-estimate variance, which matters more, not less, when the batch contains only a handful of episodes.

- **`entropy_coeff`: $0.0 \rightarrow 0.01$.** Directly targets mechanism (2): introduces continued exploration pressure proportional to policy entropy, intended to prevent the observed collapse to -0.53 before the policy has had a chance to discover the coverage-reward signal.

- **Reward rebalancing.** Collision penalty $-20.0 \rightarrow -8.0$ (reduces the dominance of a single sparse spike over a short episode); coverage-reward weight $2.0 \rightarrow 4.0$ (densifies the signal for the metric actually evaluated, partially independent of mechanism (1)–(2) but compounding with them by reducing overall return variance).

- **Episode length and batch size: `max_steps` $150 \rightarrow 90$, train batch size $300 \rightarrow 700$.** Episode-reset overhead in a physics-in-the-loop environment (settling time after simulator reset) is fixed regardless of episode length, so shorter episodes yield more distinct episodes per unit wall-clock; a larger batch size directly increases the number of episodes contributing to each gradient step, compounding with the $\lambda = 0.95$ variance reduction. Both changes increase distinct-episode count per iteration without a proportional wall-clock increase, respecting the paper's central constraint (Section 1) that environment steps cannot simply be made cheaper.

- **Iteration budget:** $15 \rightarrow 45$**.** A secondary lever, applied only after the above changes, within an estimated wall-clock budget of 1.6–3.5 hours based on observed per-iteration timing at the new batch/episode configuration.

We explicitly considered and rejected two further interventions given the project's wall-clock budget. *Curriculum learning* (training on 1 robot before scaling to 3) would require reworking spawn and goal logic for a variable robot count and a staged training driver, at a cost of multiple engineering hours, to address a credit-assignment difficulty that the centralized critic already handles structurally – we judged this a speculative fix for a problem not diagnosed in Section 4. *Behavior-cloning warm start* from a working non-learned frontier heuristic (available in the same codebase) would require an offline dataset pipeline, a supervised pretraining loop, and reconciling the heuristic's discrete waypoint targets with the continuous action space and the centralized critic's value function – a distinct problem (cold-start exploration) from the one diagnosed here (entropy collapse from advantage-variance under a tiny batch), and correspondingly not pursued.

6 Validation Status

The correction in Section 5 is implemented in the project's training script and verified not to regress existing functionality: the unit test suite (including a centralized-critic postprocessing test and a permutation-invariance test at swarm sizes unseen during training) passes unchanged, and a companion coverage-demo pipeline sharing the same environment code was independently re-verified end-to-end after the change. **[We were unable to complete the corrected configuration's full training run in this revision: three independent launch attempts (including one after disabling PyTorch's dynamo/AOT-compile subsystem, the standard mitigation for the specific crash signature observed) terminated with an identical, reproducible native segmentation fault inside `torch.optim.adam.Adam.__init__`, confirmed by matching stack traces across all three attempts and unrelated to any training-code or hyperparameter change – i.e., a local PyTorch/CUDA installation fault, not a finding about the correction itself. We report this transparently rather than fabricate or omit the pending result. A revision of this paper will report post-correction entropy trajectory, episode-return variance, and downstream task-coverage numbers once training infrastructure is repaired; the diagnosis (Section 4) and correction rationale (Section 5) do not depend on that outcome and are reported in full here.]**

We note, as a methodological point relevant to this paper's own thesis, that this infrastructure fault is itself an instance of the operational fragility that distinguishes physics-in-the-loop training from cloud-scale, parallelized-simulator MARL research: a single-machine, single-instance training pipeline has a single point of failure that a fleet of identical parallel simulator instances does not, and debugging it competes directly for the same limited wall-clock budget that motivates this paper's sample-efficiency concern in the first place.

7 Discussion

The failure mode diagnosed here – entropy collapse from high-variance advantage estimates under a batch too small for $\lambda = 1.0$, compounded by a lack of exploration pressure and a spiky reward – is not specific to multi-robot coverage or to Gazebo; it is a generic consequence of applying PPO/MAPPO default hyperparameters, tuned in a literature that assumes large-batch training, to any setting where the environment-step budget is externally bounded by real-time physics rather than by researcher choice. We expect the same diagnosis (if not the identical numeric correction) to transfer to other physics-in-the-loop, single-instance-simulator MARL settings: any setup where `num_env_runners` cannot exceed

a small number for infrastructural reasons is exposed to the same batch-size/GAE interaction. We see the primary value of this paper as making the diagnosis and correction explicit and mechanistically grounded, rather than leaving practitioners to rediscover it empirically, run by expensive run, under a sample budget that makes empirical hyperparameter search itself costly.

8 Conclusion

Under a hard, wall-clock-bounded sample budget of thousands rather than millions of environment steps – the realistic regime for physics-in-the-loop multi-robot training without dedicated parallel-simulation infrastructure – default PPO/MAPPO hyperparameters tuned for much larger batch regimes produce a specific, reproducible failure: high-variance advantage estimates from $\lambda = 1.0$, compounded by zero exploration pressure, collapsing policy entropy before the policy has seen enough data to learn the task. We diagnosed this failure mechanistically in a real multi-robot coverage system, derived a targeted correction (reduced λ, nonzero entropy bonus, reward rebalancing, and episode/batch sizing that increases distinct-episode count without increasing wall-clock), and report both the diagnosis and the correction in full. Completing the correction's empirical validation was blocked in this revision by an infrastructure fault unrelated to the training methodology, reported transparently rather than concealed; we consider the mechanistic diagnosis and grounded correction, independently useful to practitioners facing the same constraint, to stand on their own pending that validation.

References

[1] Jung In Kim, Young Jae Lee, Jongkook Heo, Jinhyeok Park, Jaehoon Kim, Sae Rin Lim, Jinyong Jeong, and Seoung Bum Kim. Sample-efficient multi-agent reinforcement learning with masked reconstruction. *PLoS One*, 2023. PMC10501567.

[2] Eduardo Sebastian, Thai Duong, Nikolay Atanasov, Eduardo Montijano, and Carlos Sagues. Physics-informed multi-agent reinforcement learning for distributed multi-robot problems, 2024. arXiv:2401.00212.

[3] Haojie Shi, Tingguang Li, Qingxu Zhu, Jiapeng Sheng, Lei Han, and Max Q.-H. Meng. An efficient model-based approach on learning agile motor skills without reinforcement, 2024. arXiv:2403.01962.

[4] Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] Yuang Zhang, Yunlong Song, Zhihao He, Zelin Ni, Kangyu Wang, Tianchi Liu, Yu Hu, Feng Yu, Danping Zou, and Weiyao Lin. Asymmetric physics enables efficient learning in quadrupedal robot swarms, 2026. arXiv:2606.23153.